Demolab

An opinionated lab notebook for computational science.

Eoin Murray · 5 July 2026

An observation

- In my circles, **software engineers** use coding agents like crazy.
- **Academics**, far less so.
- **demolab** is an attempt to close that gap.

Coding agents

- AI assistants that read a repo and **do the work**, not just autocomplete.
- Run in your terminal or editor: read files, run commands, edit code, verify.
- You drive them in **plain language**; they follow instructions in the repo.
- Prominent ones:
  - **Claude Code** (Anthropic)
  - **Cursor**
  - **GitHub Copilot** (agent mode)
  - **aider/pi**
  - **Gemini CLI** (Google)
- demolab is **operated by** one: no web dev, no build config.

Strengths of coding agents

- Came into their own in **December 2025**: reliable enough to trust with real work.
- They can **code**: read a repo, run commands, edit, test, and verify.
- They can **write**: prose, docs, and structured content.
- They can **search the literature**: synthesize sources, far better than a search engine.
- One agent can carry a whole project: code **and** the write-up around it.
- **At least 5× my old coding productivity**, and I can take on more than before.
- That's the bet demolab makes: hand it the loop, keep the science yours.

TODO: concrete examples would help here, a before/after breakdown of a real task.

Weaknesses of coding agents

- **Confidently wrong**: fabricate APIs, numbers, and citations.
- **No memory**: forget context between sessions unless it's written down.
- **Weak at judgement**: won't tell you if the experiment itself is a bad idea.
- **Drift**: left alone, code and prose fall out of sync, the classic failure.
- **Noise**: over-produce: verbose output, churn, changes you didn't ask for.
- Need **guardrails**: a fixed structure and checks, not blind trust.

It's a powerful tool that requires guardrails and constant supervision.

Weaknesses *of using* coding agents

- **Skill atrophy & brainrot**: outsource the thinking and you stop understanding your own code.
- **Attention span**: its a slot machine.
- **Review & verification burden**: the work shifts to reading and checking every claim, number, and citation.
- **Cost**: tokens and subscriptions add up on real workloads.
- **Privacy**: your code and data leave your machine.

The shape of a demolab repo

```
tools/           the science — models & solvers
experiments/     the runners — expNNN.py + playground.py
writings/        the writeups — .typ per entry, by id
artifacts/       the record — data/ + pdfs/ (committed)
demolab.yaml     branding + collections (optional)
demolab-engine/ the engine (black box) — build · runbooks · guides
temp/            run scratch (gitignored)
```

Driven by a handful of tasks

```
task install          set up (Python deps via uv)
task run -- exp000    run an experiment end-to-end
task dev              serve the site, live-reload on save
task test             run the test suite
```

What you publish

- One `task build` → a **static site**, a **PDF** per entry, and a single **book** PDF.
- The site organises itself: entries grouped into **collections**, drilling home → collection → entry, plus an all-entries index.
- Figures inline, videos play, math as real **MathML**, every number read from the run, so nothing drifts.
- A GitHub Action deploys it to **Pages** on every push: free hosting, a stable link.

The core rules

- **Science in tools; stories in experiments**: tools hold reusable computation, runners run them and write it up.
- **One experiment = a runner (`.py`) + a writeup (`.typ`)**: same id, one result.
- **Tools emit data, not plots**: the runner renders the figure from the data.
- **Tools and runners never import each other**: they talk only through files.
- **Numbers are read from the run, never typed**, so they can't drift.
- **Every result carries its git commit**: provenance by construction.

Use any stack you want

- Ships set up for **Python**: `uv`, tools + runners as `.py`, `pytest`.
- But the contract is **files, not a language**: a tool just writes `numbers.json` + data.
- So switching is a conversation: **"migrate the stack to MATLAB"**, and the agent follows a runbook.
- Works for **MATLAB · R · Julia · Octave**: Typst publishing and the contract stay put.
- Keep Python just for plotting, or drop it entirely: your call.

Getting started

The only thing you install is a **coding agent** (Claude Code, Cursor, aider, …).

1. Open it in a new, empty folder.
2. Tell it: **“Go to github.com/eoinmurray/demolab, read the README, and set it up here.”**
3. Approve as it installs the toolchain and starts the demo: it hands you a live URL.
4. Say what you want to compute: it scaffolds, runs, and publishes your first experiment.

Already have a codebase?

Say **“migrate my code”**, and the agent follows a runbook built on three principles:

- **One experiment at a time**: get one publishing end to end before the next.
- **Wrap, don’t rewrite**: the tool is a thin adapter that calls your functions; your science is untouched.
- **One environment**: your deps fold into the project (`uv add`), no parallel setup.

Each migrated experiment publishes with figures + numbers that match your original output.

Stay current

The engine is a **black box** you never edit, so upstream improvements just drop in.

- `demolab-engine/` is pure upstream; your branding (`demolab.yaml`), content, and deps live **outside** it.
- Say **"update demolab"**: the agent vendor-copies the latest engine, leaving your work untouched.
- New features land without a merge conflict in sight.